Collaboration Details:

I would like to partner with Skilvul to deliver a highly practical, cutting-edge technical training program for your advanced computer science students and alumni community. I will personally act as the lead instructor and trainer for this initiative.

- **The Topic:** Modern Concurrency in C++20: RAII Threads (`std::jthread`) and Cooperative Cancellation (`std::stop_token`).
- **The Issue/Challenge:** Traditional multithreading via legacy `std::thread` requires tedious manual thread lifecycle tracking and explicit joining. Forgetting to join or detach causes application runtime crashes, resource leaks, or dangling processes. Additionally, before C++20, building cross-thread interruption mechanisms required extensive boilerplate code with custom atomic flags, introducing synchronization bugs and data race liabilities.
- **The Case/Value Proposition:** C++20 standardizes foolproof thread safety using RAII principles and native cancellation frameworks. Learning these native tools equips Skilvul graduates to write highly optimized, performant software for demanding industries like game development, backend cloud infrastructure, and low-latency financial systems without relying on heavy external dependencies.
- **The Practical Application:** Students will apply these concepts by building an asynchronous, thread-safe Logging Engine. This project serves as an ideal hands-on capstone, teaching core data synchronization without overwhelming them with hyper-complex scheduling algorithms.
- **My Delivery Requirements & Formats:**
  I propose two flexible format options based on Skilvul's academic calendar and student availability:
  - *Option A (1-Week Mini-Bootcamp):* 1 hour per day for 5 consecutive days (5 hours total).
  - *Option B (1-Day Masterclass):* A single, focused 3-hour intensive workshop.

    I will handle curriculum engineering, slide decks, technical labs, and instruction. I look to Skilvul for student mobilization, platform or workspace hosting, and operational event marketing.

APPENDIX: DETAILED CURRICULUM BREAKDOWN

Option A Syllabus: 1-Week Mini-Bootcamp (1 Hour / Day)

- **Day 1: Pitfalls of Legacy Concurrency & The RAII Solution**
  - Reviewing legacy `std::thread` pitfalls (unjoined crashes, termination behavior).
  - Introduction to RAII in modern thread lifecycles.
  - Writing our first `std::jthread` and observing automatic safe joining.
- **Day 2: Cooperative Interruption Mechanics**
  - Understanding the limitation of forced thread termination.
  - Deep dive into `std::stop_token` and `std::stop_source`.
  - Implementing custom polling loops using `.stop_requested()`.
- **Day 3: Advanced Signaling and Thread Synchronization**
  - Interrupting blocking threads safely without deadlocks.
  - Integrating `std::condition_variable_any` with `std::stop_token`.
  - Designing an automated wakeup system upon thread cancellation.
- **Day 4: Capstone Architecture - The Async Logger**
  - Designing a thread-safe message queue using `std::mutex` and `std::queue`.
  - Setting up a dedicated background consumer worker utilizing `std::jthread`.
  - Writing non-blocking log-submission operations from the client thread.
- **Day 5: Capstone Implementation and Refactoring**
  - Coding the output buffer and log flushing logic.
  - Triggering automatic destruction to flush remaining logs gracefully at shutdown.
  - Code reviews, debugging common concurrency pitfalls, and open Q&A.

Option B Syllabus: 1-Day Masterclass (3-Hour Single Session)

- **Hour 1: Core Mechanics of Safe Concurrency**
  - *00:00 - 00:30:* Introduction to `std::jthread`, memory implications, and explicit vs. implicit joining.

    - *00:30 - 01:00:* Hands-on lab exploring `std::stop_token` polling and signaling via `.request_stop()`.
- **Hour 2: Interruption & Shared Data Boundaries**
    - *01:00 - 01:30:* Integrating `std::condition_variable_any` for instantaneous cancellation handling.
    - *01:30 - 02:00:* Designing a thread-safe queue interface using modern locks (`std::unique_lock`).
- **Hour 3: Building the Production-Ready Async Logger**
    - *02:00 - 02:40:* Live-coding lab combining `std::jthread`, `stop_token`, and the locked queue into an asynchronous logger.
    - *02:40 - 03:00:* Verifying leak-free teardowns, managing exit states, and concluding Q&A.